

家具系统重构方案

版本：v0.1

日期：2026-07-06

状态：设计阶段

一、现状分析

1.1 当前问题

所有可放置实体（面包柜、椅子、桌子）的坐标由 **id** → **数学公式** 硬编码计算：

```
// Cabinet: 位置完全被公式锁死
x = (CabinetID / 12) * 200 + 100 * (CabinetID % 2) + 20;
y = (CabinetID / 2 % 6) * 40 + 200;

// Chair: 同样不可配置
x = (ChairID / 8) * 200 + 450 + 80 * (ChairID % 2);
y = (ChairID / 2 % 4) * 100;

// Desk
x = (DeskID / 4) * 200 + 490;
y = (DeskID % 4) * 100;
```

问题	后果
坐标与 id 强耦合	无法增删单个家具，必须保持 id 连续
纯数学公式	无法让玩家自定义摆放
无碰撞检测	新增家具可能与现有物体重叠
无区域限制	桌子可以放到厨房，面包柜放到用餐区
不可存档	位置不是数据，无法持久化自定义布局
扩展性为零	每加一种新家装都要写新公式

1.2 当前实体一览

实体	类名	数量	所属场景区域
面包柜	Cabinet	≤24	展示区（左侧）
椅子	Chair	16	用餐区（右侧）
桌子	Desk	8	用餐区（右侧）
收银台	Register	1	固定位置

实体	类名	数量	所属场景区域
厨师	Cook	动态	厨房区
服务员	StoreServer	动态	全局移动

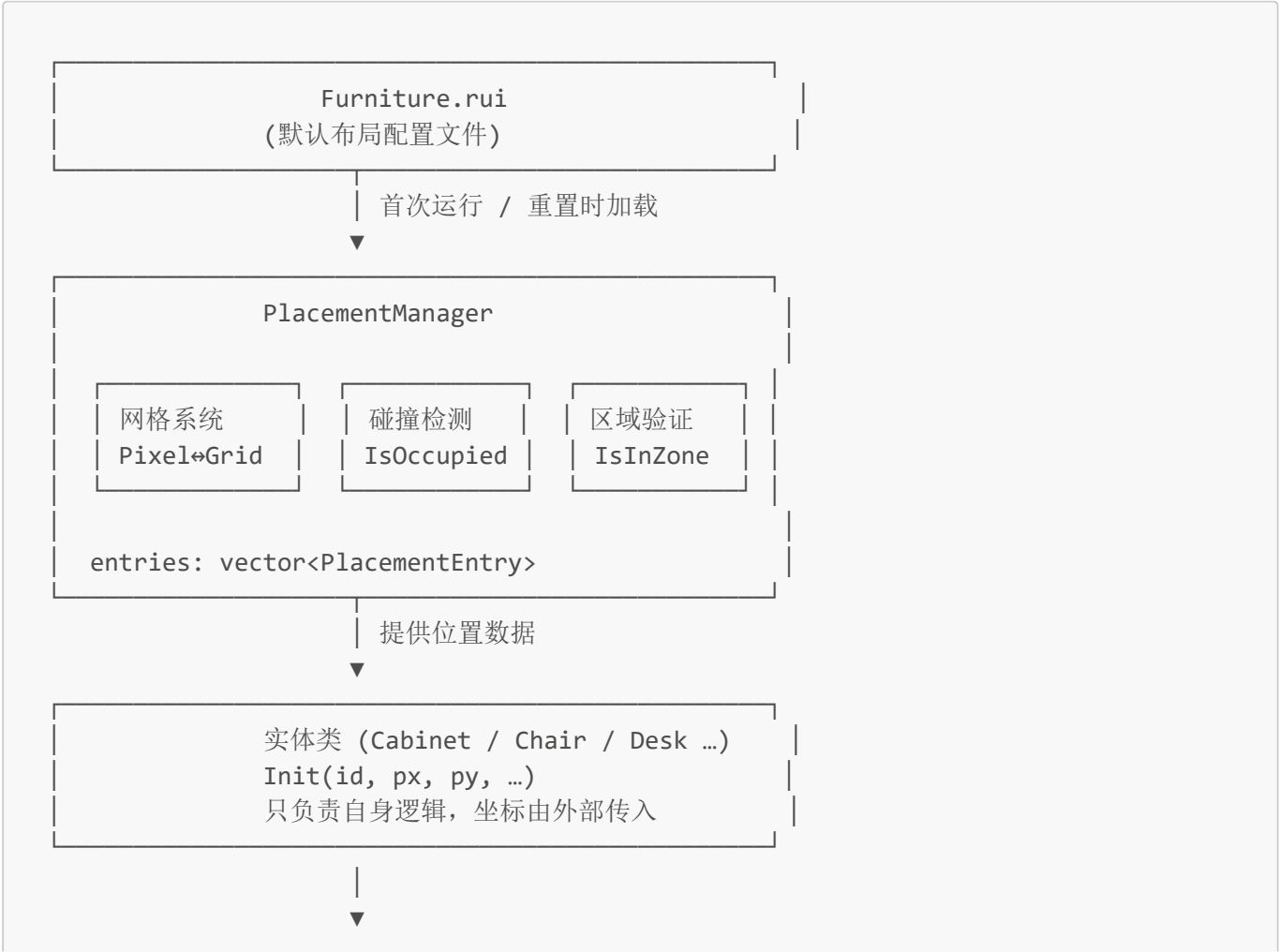
1.3 未来需求

- 玩家可拖拽摆放家具
- 支持环境装饰（植物、地毯、灯具、墙饰等）
- 多区域管理（厨房区、展示区、用餐区、VIP区...）
- 自定义布局可存档 & 分享
- 潜在的对战/竞技场景（CompetitionScene）中的评分系统

二、设计目标

1. 数据驱动：位置从配置文件读取，不依赖公式
2. 可扩展：新增家具类型只需注册，不动核心逻辑
3. 可存档：自定义布局随游戏进度持久化
4. 防非法放置：碰撞检测 + 区域验证
5. 向后兼容：首次运行自动生成默认布局文件

三、整体架构



save/Furniture.rui
(玩家自定义布局存档)

四、核心数据结构

4.1 网格坐标

```
struct GridPos
{
    int col;    // 列 (0 ~ GridCols-1)
    int row;    // 行 (0 ~ GridRows-1)
};

// 网格参数
namespace GridConfig
{
    const int CellSize = 64;    // 每格像素 (与实体贴图尺寸一致)
    const int Cols      = 12;    // 800 / 64 ≈ 12.5 → 取 12
    const int Rows      = 9;     // 600 / 64 ≈ 9.3  → 取 9
    const int OffsetX   = 0;    // 全局水平偏移
    const int OffsetY   = 0;    // 全局垂直偏移
}
```

4.2 放置条目

```
enum class FurnitureType
{
    Cabinet    = 0,    // 面包柜
    Chair      = 1,    // 椅子
    Desk       = 2,    // 桌子
    Decoration  = 3,    // 环境装饰
    // 未来扩展:
    // Rug      = 4,    地毯
    // Light    = 5,    灯具
    // Plant    = 6,    植物
    // WallArt  = 7,    墙饰
};

struct PlacementEntry
{
    int          id;        // 实体唯一ID (同类型内自增)
    FurnitureType type;     // 家具类型
    GridPos      pos;      // 网格坐标
    bool         flipped;   // 是否水平翻转
    int          zone;     // 所属区域
}
```

```
// 转为像素坐标
void ToPixel(int& outX, int& outY) const
{
    outX = pos.col * GridConfig::CellSize + GridConfig::OffsetX;
    outY = pos.row * GridConfig::CellSize + GridConfig::OffsetY;
}
};
```

4.3 区域规则

```
struct ZoneRule
{
    int            zoneId;
    std::string    name;           // "厨房" / "展示区" / "用餐区"
    SDL_Rect       pixelBounds;    // 像素边界
    std::vector<int> allowedTypes; // 允许放置的家具类型列表
};
```

预设区域示例：

区域	边界	允许类型
厨房 (0)	(0, 0, 320, 200)	Decoration
展示区 (1)	(0, 200, 320, 400)	Cabinet, Decoration
用餐区 (2)	(320, 0, 480, 600)	Chair, Desk, Decoration

五、文件格式

5.1 默认布局：resources/Furniture.rui

发行时内置，首次运行或"重置布局"时读取：

```
# Furniture.rui — 默认家具布局
# 格式：类型 实体ID 列 行 翻转 区域
# 类型：0=面包柜 1=椅子 2=桌子 3=装饰
# 翻转：0=正常 1=水平镜像
# 区域：0=厨房 1=展示区 2=用餐区

# === 面包柜（展示区） ===
0 0 0 3 0 1
0 1 1 3 0 1
0 2 2 3 0 1
0 3 3 3 0 1
# ... 共24个

# === 桌子（用餐区） ===
2 0 7 0 0 2
```

```

2 1 7 1 0 2
# ... 共8个

# === 椅子 (用餐区) ===
1 0 6 0 0 2
1 1 8 0 1 2
# ... 共16个

# === 收银台 (展示区, 固定位置) ===
# type=3 装饰类, 不允许移动
3 0 4 1 0 1

```

5.2 玩家存档: `save/Furniture.rui`

格式与默认文件完全相同。`PlacementManager` 优先读 `save/`, 不存在时回退到 `resources/`。

5.3 区域定义: `resources/Zones.rui`

```

# 格式: 区域ID 名称 x y w h
# 后面跟允许的家具类型列表, -1结束
0 厨房    0    0 320 200
3 -1
1 展示区  0 200 320 400
0 3 -1
2 用餐区  320 0 480 600
1 2 3 -1

```

六、PlacementManager 接口

```

class PlacementManager
{
public:
    // ===== 加载 / 保存 =====
    void LoadDefaults();           // 读 resources/Furniture.rui
    void LoadFromSave();          // 读 save/Furniture.rui
    void SaveToFile();             // 写 save/Furniture.rui
    void ResetToDefaults();        // 恢复默认布局

    // ===== 查询 =====
    std::vector<PlacementEntry> GetByType(FurnitureType type) const;
    const PlacementEntry*       FindById(int id, FurnitureType type) const;
    GridPos                     PixelToGrid(int px, int py) const;

    // ===== 修改 =====
    bool TryPlace(int id, FurnitureType type, GridPos pos, bool flipped);
    bool TryMove(int id, FurnitureType type, GridPos newPos);
    bool TryRemove(int id, FurnitureType type);

```

```
// ===== 验证 =====
bool IsOccupied(GridPos pos) const;           // 该格是否已有物体
bool IsInBounds(GridPos pos) const;           // 是否在屏幕内
bool IsInZone(GridPos pos, int zoneId) const; // 是否在指定区域内
bool CanPlaceType(FurnitureType type, int zoneId) const; // 类型可否放该区域
bool IsOverlapping(GridPos pos, int sizeW, int sizeH) const; // 多格物体碰撞

// ===== 区域 =====
const std::vector<ZoneRule>& GetZones() const;

private:
    std::vector<PlacementEntry> m_entries;
    std::vector<ZoneRule>       m_zones;
    int                         m_nextId[10]; // 每种类型的自增ID计数器
};
```

七、实体类改造

7.1 当前 → 改造后

类	当前 Init 签名	改造后 Init 签名
Cabinet	InitCabinet(id, did, number)	InitCabinet(id, did, number, pixelX, pixelY)
Chair	InitChair(id)	InitChair(id, pixelX, pixelY, flipped)
Desk	initDesk(id)	InitDesk(id, pixelX, pixelY)

7.2 调用方式变化

```
// 改前 — GameScene::onEnter()
for (int i = 0; i < 16; i++)
{
    Chair chair;
    chair.InitChair(i);           // 位置在 Init 内部算
    chairs.push_back(chair);
}

// 改后
auto& chairEntries = placementManager.GetByType(FurnitureType::Chair);
for (auto& entry : chairEntries)
{
    Chair chair;
    int px, py;
    entry.ToPixel(px, py);
    chair.InitChair(entry.id, px, py, entry.flipped);
    chairs.push_back(chair);
}
```

八、存档改造

在 `GameSerializer` 中增加家具位置读写：

```
# save/Furniture.rui （保存时完整写入，加载时完整读取）
0 0 0 3 0 1
0 1 1 3 0 1
...
```

加载流程：

```
GameScene::onEnter()
├─ customerManager / materialManager 初始化
├─ GameSerializer::Load()           // 读经济、时间、员工数据
├─ placementManager.LoadFromSave()  // 优先读 save/Furniture.rui
│   └─ 不存在则 LoadDefaults()    // 回退到 resources/Furniture.rui
└─ 遍历 placementManager entries → 初始化各实体
```

九、UI 编辑模式（远期）

9.1 进入编辑模式

- 点击图标栏新增的"编辑布局"按钮
- 或长按某个家具 2 秒进入拖拽状态

9.2 编辑交互

操作	效果
点击家具	高亮 + 显示可移动范围
拖拽到空格	预览半透明 + 绿色（合法） / 红色（非法）
松手	确认放置或弹回原位
右键家具	弹出菜单：旋转 / 移除 / 取消
工具栏	选择要添加的家具类型 → 点击空格放置

9.3 视觉反馈

```
合法放置：绿色半透明轮廓
非法放置：红色半透明 + 抖动
已占用格：灰色遮罩
可放置格：轻微高亮
```

十、分阶段实施计划

阶段一：数据结构 + 默认配置 ★ 核心

任务	涉及文件
新增 PlacementEntry + GridPos 数据结构	include/RUI_PlacementData.h
新增 PlacementManager 类	include/RUI_PlacementManager.h src/
编写默认布局 resources/Furniture.rui	新文件
编写区域配置 resources/Zones.rui	新文件
改造 Cabinet::InitCabinet 接受像素坐标	include/RUI_Cabinet.h
改造 Chair::InitChair 接受像素坐标	include/RUI_Chair.h
改造 Desk::initDesk 接受像素坐标	include/RUI_Chair.h

阶段二：GameScene 接入

任务	涉及文件
GameScene 增加 PlacementManager 成员	include/RUI_GameScene.h
onEnter() 从 manager 读取位置初始化实体	src/RUI_GameScene.cpp
GameSerializer 增加家具存档读写	include/RUI_GameSerializer.h src/

阶段三：编辑功能 ★ 体验飞跃

任务	涉及文件
新增"编辑布局"图标	include/RUI_Icon.h
拖拽逻辑：选中 → 预览 → 确认/取消	src/RUI_GameScene.cpp
碰撞检测 + 区域验证集成	PlacementManager
编辑模式 UI（工具栏、网格线）	onRender() 新增渲染层

阶段四：扩展家具类型

任务	说明
新增 Decoration 实体类	纯视觉，提升"环境评分"
新增家具贴图资源	resources/decoration/
扩展 FurnitureType 枚举	注册新类型
CompetitionScene 接入放置系统	评分维度之一

十一、潜在扩展方向

- **家具商店**：用游戏币购买新家具，解锁后可放置
- **主题套装**：日式 / 欧式 / 赛博朋克风格一键切换
- **环境评分**：装饰数量 × 搭配度 → 影响顾客满意度
- **布局分享码**：将布局序列化为短字符串，方便玩家交流
- **多房间**：店铺扩张后解锁新房间，每间独立布局
- **员工站位**：厨师和服务员的工作位置也可配置